

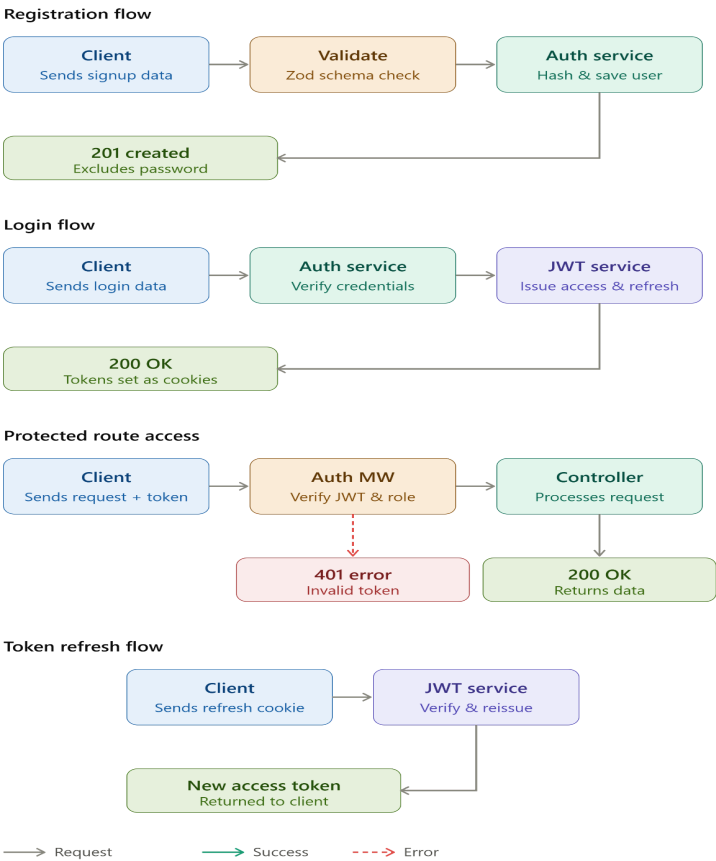
School Sphere - Product Requirements Document (PRD)

Project Overview

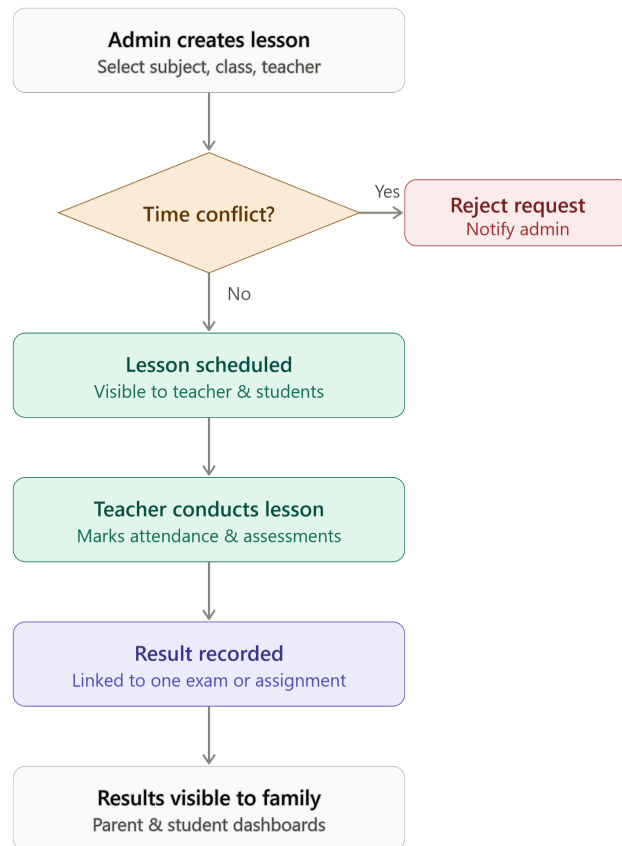
Name: School Sphere Backend **Type:** RESTful API for School Management Platform **Tech Stack:** Node.js, Express.js, TypeScript, Prisma ORM, PostgreSQL, Cloudinary (image storage)

Visual Documentation

JWT Authentication Flow



Business Logic Flow



1. Core Functionality

1.1 User & Role Management

- **Roles:** admin, teacher, parent, student
- **Authentication:** JWT-based auth with access & refresh tokens
- **Profile:** Name, surname, username, email, phone, address, image, blood type, sex, birthday (role-dependent fields)

1.2 Academic Structure Management

- **CRUD Operations:** Grades, Classes, Subjects
- **Class Properties:** name, capacity, supervisor (teacher), grade
- **Subject Properties:** name, assigned teachers

1.3 Lesson Scheduling

- **Create Lesson:** Admin schedules a lesson for a subject, class, and teacher
- **Properties:** name, day (Mon–Fri), startTime, endTime
- **Conflict Detection:** No overlapping lessons for the same teacher or class

1.4 Assessment System

- **Exams & Assignments:** Created per lesson by the assigned teacher
- **Results:** Score entry tied to a student and either one exam or one assignment
- **Flow:** Teacher creates exam/assignment → records results → parents/students view scores

1.5 Attendance System

- **Mark Attendance:** Teacher marks present/absent per student, per lesson, per date
- **View Attendance:** Students/parents view attendance history

1.6 Communication System

- **Events:** School-wide or class-specific, with start/end time
- **Announcements:** School-wide or class-specific, dated

1.7 Image Upload System

- **Profile Images:** Teacher and Student `img` field populated via upload
- **Flow:** Client sends `multipart/form-data` → Multer parses file → Cloudinary stores it → secure URL saved to the record
- **Constraints:** Image only (jpg/png/webp), max file size enforced (e.g., 2MB), old image replaced (not accumulated) on re-upload

2. Product Data Model (PRD Style)

Admin

Description Represents platform administrators who manage all academic and user data.

Attributes

- ID (unique identifier)
- Username (unique)

Relationships

- Has no direct data relationships; holds full管理 (management) access across the system
-

2 Teacher

Description Represents a staff member who teaches subjects, conducts lessons, and may supervise a class.

Attributes

- ID (unique identifier)
- Username (unique)
- Name, Surname
- Email (optional, unique)
- Phone (optional, unique)
- Address
- Profile image (optional)
- Blood type
- Sex (Male / Female)
- Birthday
- Created date

Relationships

- A teacher can teach **multiple subjects** (many-to-many)
 - A teacher can conduct **multiple lessons**
 - A teacher can supervise **multiple classes**
-

3 Parent

Description Represents a guardian linked to one or more students.

Attributes

- ID (unique identifier)
- Username (unique)
- Name, Surname
- Email (optional, unique)
- Phone (required, unique)
- Address

- Created date

Relationships

- A parent can have **multiple students**
-

4 Student

Description Represents an enrolled learner linked to a parent, class, and grade.

Attributes

- ID (unique identifier)
- Username (unique)
- Name, Surname
- Email (optional, unique)
- Phone (optional, unique)
- Address
- Profile image (optional)
- Blood type
- Sex (Male / Female)
- Birthday
- Created date

Relationships

- A student belongs to **one parent**
 - A student belongs to **one class**
 - A student belongs to **one grade**
 - A student can have **multiple attendance records**
 - A student can have **multiple results**
-

5 Grade

Description Represents a grade level (e.g., Grade 5) grouping multiple classes and students.

Attributes

- ID (unique identifier)
- Level (unique)

Relationships

- A grade can have **multiple students**
 - A grade can have **multiple classes**
-

6 Class

Description Represents a specific class/section within a grade.

Attributes

- ID (unique identifier)
- Name (unique)
- Capacity
- Created date

Relationships

- A class belongs to **one grade**
 - A class may have **one supervising teacher** (optional)
 - A class can have **multiple students**
 - A class can have **multiple lessons**
 - A class can have **multiple events**
 - A class can have **multiple announcements**
-

7 Subject

Description Represents an academic subject (e.g., Math, Science).

Attributes

- ID (unique identifier)
- Name (unique)

Relationships

- A subject can be taught by **multiple teachers**
 - A subject can have **multiple lessons**
-

8 Lesson

Description Represents a scheduled class session for a subject, taught by a teacher, on a specific day and time.

Attributes

- ID (unique identifier)
- Name
- Day (Monday–Friday)
- Start time
- End time

Relationships

- A lesson belongs to **one subject**
 - A lesson belongs to **one class**
 - A lesson belongs to **one teacher**
 - A lesson can have **multiple exams**
 - A lesson can have **multiple assignments**
 - A lesson can have **multiple attendance records**
-

9 Exam

Description Represents a scheduled exam tied to a lesson.

Attributes

- ID (unique identifier)
- Title
- Start time
- End time

Relationships

- An exam belongs to **one lesson**
 - An exam can have **multiple results**
-

10 Assignment

Description Represents a take-home task tied to a lesson.

Attributes

- ID (unique identifier)
- Title
- Start date
- Due date

Relationships

- An assignment belongs to **one lesson**
 - An assignment can have **multiple results**
-

11 Result

Description Represents a student's score for either an exam or an assignment.

Attributes

- ID (unique identifier)
- Score

Relationships

- A result belongs to **one student**
 - A result belongs to **one exam OR one assignment** (mutually exclusive)
-

12 Attendance

Description Represents a student's presence record for a specific lesson on a specific date.

Attributes

- ID (unique identifier)
- Date
- Present (boolean)

Relationships

- An attendance record belongs to **one student**
- An attendance record belongs to **one lesson**

13 Event

Description Represents a school or class-specific event.

Attributes

- ID (unique identifier)
- Title
- Description
- Start time
- End time

Relationships

- An event may belong to **one class** (optional — null means school-wide)
-

14 Announcement

Description Represents a school or class-specific announcement.

Attributes

- ID (unique identifier)
- Title
- Description
- Date

Relationships

- An announcement may belong to **one class** (optional — null means school-wide)
-

Relationship Summary

- One **Parent** → can have many **Students**
- One **Grade** → can have many **Students** and many **Classes**
- One **Class** → belongs to one **Grade**, may have one **Teacher** (supervisor), has many **Students**
- One **Teacher** → can teach many **Subjects** (M:N), conduct many **Lessons**, supervise many **Classes**
- One **Subject** → can be taught by many **Teachers** (M:N), has many **Lessons**

- One **Lesson** → belongs to one **Subject**, one **Class**, one **Teacher**
 - One **Lesson** → can have many **Exams**, **Assignments**, **Attendance** records
 - One **Exam** → belongs to one **Lesson**, has many **Results**
 - One **Assignment** → belongs to one **Lesson**, has many **Results**
 - One **Result** → belongs to one **Student**, and to one **Exam** OR one **Assignment**
 - One **Attendance** → belongs to one **Student** and one **Lesson**
 - One **Event** → may belong to one **Class** (or none, i.e., school-wide)
 - One **Announcement** → may belong to one **Class** (or none, i.e., school-wide)
-

Business Rules

- Only users with role **Teacher** can create exams, assignments, and mark attendance for their own lessons
 - A **Result** must reference either an exam or an assignment, never both, never neither
 - A **Lesson** cannot overlap in time with another lesson for the same teacher or the same class
 - A **Class** cannot enroll more students than its defined capacity
 - A **Student's** grade must match the grade of the class they are enrolled in
 - **Attendance** can only be marked once per student, per lesson, per date
 - **Events** and **Announcements** with no class assigned are visible to the entire school
 - Deleting a **Parent**, **Teacher**, **Class**, **Grade**, or **Subject** with dependent records must be blocked or explicitly cascaded
-

3. API Endpoints

Authentication

Method	Endpoint	Description
POST	<code>/api/v1/auth/register</code>	Register new user (admin-invoked)
POST	<code>/api/v1/auth/login</code>	Login with username/password
POST	<code>/api/v1/auth/refresh-token</code>	Refresh access token

Users (Admin, Teacher, Parent, Student)

Method	Endpoint	Description	Auth
--------	----------	-------------	------

POST	<code>/api/v1/teachers</code>	Create teacher	Admin
GET	<code>/api/v1/teachers</code>	Get all teachers (search/filter/paginate)	Admin
GET	<code>/api/v1/teachers/:id</code>	Get teacher by ID	Admin, Teacher (self)
PATCH	<code>/api/v1/teachers/:id</code>	Update teacher	Admin
DELETE	<code>/api/v1/teachers/:id</code>	Delete teacher	Admin
POST	<code>/api/v1/parents</code>	Create parent	Admin
GET	<code>/api/v1/parents</code>	Get all parents	Admin
GET	<code>/api/v1/parents/:id</code>	Get parent by ID	Admin, Parent (self)
PATCH	<code>/api/v1/parents/:id</code>	Update parent	Admin
DELETE	<code>/api/v1/parents/:id</code>	Delete parent	Admin
POST	<code>/api/v1/students</code>	Create student	Admin
GET	<code>/api/v1/students</code>	Get all students (search/filter/paginate)	Admin, Teacher
GET	<code>/api/v1/students/:id</code>	Get student by ID	Admin, Teacher, Parent (own child), Student (self)
PATCH	<code>/api/v1/students/:id</code>	Update student	Admin

DELETE	/api/v1/students/:id	Delete student	Admin
--------	----------------------	----------------	-------

Academic Structure

Method	Endpoint	Description	Auth
POST	/api/v1/grades	Create grade	Admin
GET	/api/v1/grades	Get all grades	Public (authenticated)
POST	/api/v1/classes	Create class	Admin
GET	/api/v1/classes	Get all classes	Public (authenticated)
GET	/api/v1/classes/:id	Get class by ID	Public (authenticated)
PATCH	/api/v1/classes/:id	Update class	Admin
DELETE	/api/v1/classes/:id	Delete class	Admin
POST	/api/v1/subjects	Create subject	Admin
GET	/api/v1/subjects	Get all subjects	Public (authenticated)
PATCH	/api/v1/subjects/:id	Update subject	Admin
DELETE	/api/v1/subjects/:id	Delete subject	Admin

Lessons

Method	Endpoint	Description	Auth
POST	/api/v1/lessons	Create lesson (conflict-checked)	Admin

GET	<code>/api/v1/lessons</code>	Get all lessons (filter by class/teacher/day)	Admin, Teacher, Parent, Student
GET	<code>/api/v1/lessons/:id</code>	Get lesson by ID	Admin, Teacher, Parent, Student
PATCH	<code>/api/v1/lessons/:id</code>	Update lesson	Admin
DELETE	<code>/api/v1/lessons/:id</code>	Delete lesson	Admin

Exams & Assignments

Method	Endpoint	Description	Auth
POST	<code>/api/v1/exams</code>	Create exam	Teacher, Admin
GET	<code>/api/v1/exams</code>	Get all exams	All roles
PATCH	<code>/api/v1/exams/:id</code>	Update exam	Teacher (own), Admin
DELETE	<code>/api/v1/exams/:id</code>	Delete exam	Teacher (own), Admin
POST	<code>/api/v1/assignments</code>	Create assignment	Teacher, Admin
GET	<code>/api/v1/assignments</code>	Get all assignments	All roles
PATCH	<code>/api/v1/assignments/:id</code>	Update assignment	Teacher (own), Admin
DELETE	<code>/api/v1/assignments/:id</code>	Delete assignment	Teacher (own), Admin

Results

Method	Endpoint	Description	Auth
POST	<code>/api/v1/results</code>	Record a result	Teacher, Admin

GET	<code>/api/v1/results</code>	Get results (filter by student/exam/assignment)	Admin, Teacher, Parent (own child), Student (self)
PATCH	<code>/api/v1/results/:id</code>	Update result	Teacher (own), Admin
DELETE	<code>/api/v1/results/:id</code>	Delete result	Admin

Attendance

Method	Endpoint	Description	Auth
POST	<code>/api/v1/attendances</code>	Mark attendance	Teacher, Admin
GET	<code>/api/v1/attendances</code>	Get attendance records (filter by student/lesson/date)	Admin, Teacher, Parent (own child), Student (self)
PATCH	<code>/api/v1/attendances/:id</code>	Update attendance	Teacher (own lesson), Admin

Image Upload

Method	Endpoint	Description	Auth
POST	<code>/api/v1/teachers/:id/image</code>	Upload/replace teacher profile image	Admin, Teacher (self)
POST	<code>/api/v1/students/:id/image</code>	Upload/replace student profile image	Admin
DELETE	<code>/api/v1/teachers/:id/image</code>	Remove teacher profile image	Admin, Teacher (self)
DELETE	<code>/api/v1/students/:id/image</code>	Remove student profile image	Admin

Events & Announcements

Method	Endpoint	Description	Auth
--------	----------	-------------	------

POST	<code>/api/v1/events</code>	Create event	Admin, Teacher (own class)
GET	<code>/api/v1/events</code>	Get all events	All roles
PATCH	<code>/api/v1/events/:id</code>	Update event	Admin
DELETE	<code>/api/v1/events/:id</code>	Delete event	Admin
POST	<code>/api/v1/announcements</code>	Create announcement	Admin, Teacher (own class)
GET	<code>/api/v1/announcements</code>	Get all announcements	All roles
PATCH	<code>/api/v1/announcements/:id</code>	Update announcement	Admin
DELETE	<code>/api/v1/announcements/:id</code>	Delete announcement	Admin

4. Project Structure

```

src/
├── config/index.ts      # Environment config
├── lib/
│   ├── prisma.ts       # Prisma client instance
│   └── cloudinary.ts    # Cloudinary SDK config
├── errors/
│   ├── AppError.ts     # Custom error class
│   ├── handlePrismaError.ts # Prisma error handler
│   ├── handlePrismaValidationError.ts
│   └── handleZodError.ts # Zod validation errors
├── interface/
│   └── error.ts        # Error type definitions
├── middlewares/
│   ├── auth.ts         # JWT auth middleware + role guard
│   ├── upload.ts       # Multer config (memory storage, file filter, size limit)
│   ├── globalErrorHandler.ts # Global error handler
│   ├── notFound.ts     # 404 handler
│   └── validateRequest.ts # Zod validation middleware

```

```
— modules/
  — Auth/
    — auth.controller.ts
    — auth.route.ts
    — auth.service.ts
    — auth.utils.ts    # JWT helpers
    — auth.validation.ts
  — Admin/
  — Teacher/
    — teacher.constant.ts
    — teacher.controller.ts
    — teacher.interface.ts
    — teacher.route.ts
    — teacher.service.ts
    — teacher.validation.ts
  — Parent/
  — Student/
  — Grade/
  — Class/
    — class.controller.ts
    — class.route.ts
    — class.service.ts
    — class.validation.ts
  — Subject/
  — Lesson/
    — lesson.controller.ts
    — lesson.route.ts
    — lesson.service.ts  # includes conflict-check logic
    — lesson.validation.ts
  — Exam/
  — Assignment/
  — Result/
    — result.controller.ts
    — result.route.ts
    — result.service.ts  # enforces exam XOR assignment rule
    — result.validation.ts
  — Attendance/
  — Event/
  — Announcement/
— routes/index.ts      # Route aggregator
— utils/
  — catchAsync.ts      # Async error wrapper
  — sendResponse.ts    # Response formatter
— app.ts               # Express app setup
```


└── server.ts # Server entry point

5. Technical Specifications

Environment Variables

```
NODE_ENV=development
PORT=5000
DATABASE_URL=postgresql://user:password@host:5432/dbname
JWT_ACCESS_SECRET=secret_key
JWT_ACCESS_EXPIRES_IN=1d
JWT_REFRESH_SECRET=refresh_secret
JWT_REFRESH_EXPIRES_IN=7d
BCRYPT_SALT_ROUND=12
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret
```

Dependencies

Production:

- express, cors, cookie-parser, dotenv
- @prisma/client, @prisma/adapters, pg
- bcryptjs, jsonwebtoken
- zod, http-status
- multer, cloudinary, multer-storage-cloudinary

Development:

- typescript, ts-node-dev, prisma
- eslint, prettier, @types/* (including @types/multer)

Note: Stripe/payment integration is intentionally excluded — fee/payment management remains out of scope for v1 (see Future Considerations).

6. Business Logic Rules

1. **Registration:** Admin creates Teacher/Parent/Student accounts; passwords hashed with bcrypt before storing

2. **Login:** Verify credentials, return access + refresh tokens as httpOnly cookies
 3. **Auth Middleware:** Verify JWT, check user exists, validate role permissions per route
 4. **Create Lesson:** Admin only; system checks for time-overlap conflicts against the same teacher and same class before saving
 5. **Create Exam/Assignment:** Only the teacher assigned to the parent lesson (or Admin) may create it
 6. **Record Result:** Must reference exactly one of `examId` or `assignmentId`, never both; only the owning teacher or Admin may write
 7. **Mark Attendance:** Only the teacher assigned to the lesson (or Admin) may mark; one record per student/lesson/date enforced at DB level (unique composite constraint)
 8. **Scoped Visibility:** Parents can only query data for their own linked students; Students can only query their own records; Teachers are scoped to their own lessons/classes
 9. **Pagination:** All list endpoints support `page`, `limit`, `sortBy`, `sortOrder`
 10. **Search:** Students/Teachers searchable by name, class, or subject
 11. **Image Upload:** Multer validates file type/size in memory before forwarding to Cloudinary; only Admin (or the Teacher for their own profile) may upload/replace an image; the previous image is deleted from Cloudinary when replaced, not retained
-

7. Response Format

Success Response

```
{
  "success": true,
  "message": "Resource retrieved successfully",
  "meta": { "page": 1, "limit": 10, "total": 100 },
  "data": { ... }
}
```

Error Response

```
{
  "success": false,
  "message": "Error message",
  "errorSources": [
    { "path": "field_name", "message": "Specific error" }
  ],
  "stack": "..." // Only in development
}
```

8. Deployment Checklist

1. Set `NODE_ENV=production`
 2. Configure PostgreSQL (Neon, Supabase, or self-hosted)
 3. Run `npx prisma generate && npx prisma migrate deploy`
 4. Build: `npm run build`
 5. Start: `npm start`
-

9. Future Considerations (Post-v1)

- Fee/payment management module (Stripe integration)
- SMS/email notifications for attendance and announcements
- Multi-guardian support per student
- Analytics dashboard (attendance trends, grade distributions)
- File uploads for assignment submissions (student-uploaded work, distinct from v1's profile-image uploads)